TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham
Faculty of Applied Natural Sciences and Industrial Engineering
Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Master's thesis to obtain the academic degree:
Master of Engineering (M.Eng.)

Submitted by: Sooraj Sudhakaran.
Matriculation Number: XXXXXXXX
First Examiner: Prof. Dr. Matthias Górka

Cham, February 28, 2026

Abstract

Acknowledgements

List of Tables

3.1 Functional Requirements for Automatic Robot Program Generation 13
3.2 Requirement Traceability Matrix 18

List of Figures

3.1 System architecture of the automatic robot program generation framework 22

4.1 Integration of planning system with AnyRobot Interface 29

List of Abbreviations

OLP : Offline Programming
ROS : Robot Operating System
TCP : Tool Center Point
UI : User Interface
CAD : Computer-Aided Design
CAM : Computer-Aided Manufacturing
API : Application Programming Interface
DOF : Degree of Freedom
FR : Functional Requirements
NFR : Non-Functional Requirements

Contents

Abstract **I**

Acknowledgements **III**

List of Tables **V**

List of Figures **VII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 head of robot 1

2 State of the Art **2**
2.1 Manual And teach pendant based robot programming. 2
2.2 Offline programming approaches for industrial robots 3
2.3 Automatic and Rule-Based Robot Program Generation 4
2.4 Robotic Sheet-Metal Bending and Press-Brake Cells 5
2.5 Summary of the State of the Art and Identified Research Gap . 7

3 System Concept and Methodology **9**
3.1 Problem Definition and Objectives 9
3.2 Requirement Definition and Management 10
3.2.1 Functional Requirements 11
3.2.2 Non-functional Requirements 12
3.2.3 System Constraints and Assumptions 15
3.2.4 System Traceability 17
3.3 System Architecture and Design 19
3.3.1 Input and configuration layer 19
3.3.2 Geometry and Constraint Evaluation Layer 19
3.3.3 Planning and Decision Logic layer 20
3.3.4 Motion generation layer 20
3.3.5 Safety and validation layer 20
3.3.6 Architectural Rationale 21
3.4 Algorithmic Implementation and Workflow 21
3.4.1 Initialization and configurations validation 23

3.4.2 Pose Acquisition and Orientation Normalization 23
3.4.3 Grip Position Determination and Validation 24
3.4.4 Reachability Assessment and strategy selection 24
3.4.5 Motion generation and execution 25
3.4.6 Iterative Multi bend processing 25
3.4.7 opposite side handling and controlled completion 25
3.4.8 Deterministic and constraint driven proof of correctness . 26
3.4.9 State Management and Object Tracking 26

4 System Implementation and Integration 27
4.1 Planning and constraint Module Implementation 27
4.1.1 Layered Software Architecture Realization 28

References 31

1 Introduction

YSBASHFKJSDHFKfdghghjg

1.1 head of robot

XGFGF sooraj sudhakaran

2 State of the Art

2.1 Manual And teach pendant based robot programming.

The programming of the industrial robots follows a programming approach based on teach pendant programming, which means, the robot movement and motions are explicitly defined and programmed directly during the operations. The programmer subsequently uses this approach to position and then programs the robot to move to a series of specific target positions. The robot subsequently saves these positions using programming languages specific to each robot type and brand. Despite advances in programming approaches for robotics systems, this method is still commonly used to its owing to its reliability and its usefulness in interacting with real world systems [1] . However, this may require expert knowledge of the robot's kinematics, tooling, and process constraints. Minor changes in geometry, fixture placement or cell layout often involve extensive manual reprogramming, increases commissioning effort and system downtime. Therefore, teach pendant programming scales poorly in flexible manufacturing systems characterized by frequent changeovers coupled with small batch sizes. In terms of the sheet metal handling and the automation of the press brake operations, the restrictions can appear even more considerable. When working with these pieces of sheet metal that can be flexible and large in size, their handling involves further challenges that need to be addressed, revolving around reachability issues and collision avoidance. In terms of manual programming , each step of the planned process of the bending involves the precise determination by the user of the intermediary positions for each action of the manipulator necessary for the accomplishment of the task at hand .Such an approach can appear heavily dependent on the users expertise for the purpose of avoiding the iterative try and error approach that might be inevitably required during the plant operations [2] .Especially , more classical robot programming paradigms have be identified as insufficient in terms of abstraction and flexibility needs for current applications in industrial robotics, especially when high flexibility and fast reconfiguration are at stake [1] .Such an approach to programming the robot , through interaction with the teach pendant itself , tend to create program that are more dependent on the specific type of the robot used and also the cell configuration, programming style of each particular operator etc.

As a result, manual teach in pendant programming methods have traditionally been seen to be unsuitable for advanced automation tasks. that often requires increased flexibility, analyses of process knowledge, and minimization of human interventions. It is realization that sparked off interest in alternative offline programming, task level programming and even methods of automatically generating robot programs, as discussed in the sections [1]

2.2 Offline programming approaches for industrial robots

The disadvantages associated with manual programming of industrial robotic arms lead to an increase in the uptake of offline programming in industrial applications. This programming method led to running the digital working environment using digital models of robots, tools, grippers and other components used for the programming without interrupting the real production. [1]

This programming is based on Computer aided Model (CAM) also known as the geometric or CAD models, which represent the robots, workpiece and all other components. These models allow robot programs to be simulated before they implement in the real working environment. Despite being associated with the disadvantages, offline programming also has some limitations when used in industrial applications.[3]

In robot- assisted sheet Metal handling presses brake operation, offline programming is commonly used for robot programs such as sheet metal handling, bending process, pick and regrip operations for sheet in different bending stages. Using virtual simulation, engineers can evaluate, validate and verify , and can identify the if the robot's reachability, regripping positions and bending sequences while safely avoiding the unreachable path to machines and fixtures. [2]. Even though this program offers many benefits, it relies heavily on exact digital models of robot cells. If the digital model does not perfectly match the real model configurations, due to the calibration error, fixture errors or in change in tool geometry model, the robot may misbehave in kinematics. In such cases, manual re- configuration needed, engineers must re-program during the commissioning. Another limitation of this programming is that it is less suitable for the environment with frequent changeovers and high-end product variety [1] This occurs because the programs are usually created for specific tasks and robot paths for, specific part geometry and bending process. [3] In case multi side bending process, regripping and regrasping operation is required, often require the workpiece to be temporarily place in orientation table and then grasped again in a dif-

ferent configuration. This is necessary to avoid collision with and optimize the robot kinematics. In this programming, these approaches are checked by the programmer, which increases the engineering effort that leads to limiting flexibility and time consuming. [2].

In summary, this program provides clear advantages and over manual programming by reducing the on-site teaching time and enabling virtual validation robot waypoints. However, it depends heavily on exact digital model and manual definition of task sequences reduces the efficiencies in industrial applications. These limitations have motivated into high level research, task oriented automatic program generation strategies, which aim to reduce human involvement while increasing the adaptability of the system. These approaches are explained in the following sections.

2.3 Automatic and Rule-Based Robot Program Generation

To reduce further engineering effort associated with manual and offline programming such as manual programming and offline programming, research in industrial robot section has increasingly focused on automatic program generation. This in-depth research aims to derive executable high level automated tasks with respect to robotics position and geometric information, thereby minimizing direct human re programming. In contrast to traditional offline programming, where engineer defines the motion trajectories, automated programming seeks to capture the environmental details in reusable and systematic representations.[1], [3].

A prominent solution executed in this method is rule based and task level robot program generation. In such methods, predefined rules or tasks are used to resolve the high-level constraints related to kinematics, tool orientation, and process feasibility. This rule-based programming reduces the operator role and enable the robot to perform intelligently for recurring task in industrial environment [3], [4]. Several researchers have been used reusable primitives as a foundation for automatic robot programming. Rather than programming individual motions, these complex robot behaviors represent a meaningful action such as grasping, repicking, or approaching target positions. [4], [5]. Automated programs is supported by geometric and feature based representation of parts such CAD Models or simplified geometric descriptions used for detecting or identifying the features such as grasping edges for the mechanical operations, approach directions, task sequences can be processed automatically. As compared to manual and conventional programming methods, this method enables adaptations to complex prod-

uct variants. [6]. This automated program generation has also been invented for sheet metal handling and robotic press brake automation applications, where repetitive task for bending operations and geometry is usually structured. In such systems, bending sequences, grasp configurations and placement strategies are derived from the constraints and geometric models rather than manual teaching. This reduces the engineer's effort and gives highly efficient output for repeatable similar workpieces. [2], [7].

Despite these advantages, rule-based robot program generations lead to some limitations. Most systems relay simply on the assumptions regarding part geometry, fixture layouts, tooling position and trajectories generations, which lead to restriction on applicability in highly variant part processes and low volume production scenarios. As task complexity increases, the number of rules may also get increased, which makes it hard to maintain the system, difficult to scale and lead to inconsistencies. [1], [4]. Regripping operations is one of the most highlighting problems in the sheet metal handling process. For multi-step bending , robot need to place the workpiece on temporary place called orientation table , which is used for the repegging and repositioning the workpiece for next sequence bending .Although there exist some regripping strategies , these are particularly for some specific tools and specific robot cells. [2], [7]. As a result, they do not support other setups well and limit the chances of automation.

Automatic program generation has also been combined with optimization-based methods, where trajectory quality is considered during motion generation. Such methods improve the execution performance with generating trajectory plans in robot handling tasks [8].

In conclusion, automatic and rule-based programs subsequently reduce the engineer's effort and increase consistency for complex task specifications. However, some approaches are still considered as the limitations, limited adaptability to process different variations about geometry and handling. These limitations give motivation to develop specific solutions with high degree automation, specifically for the sheet metal bending, which is the focus of this research.

2.4 Robotic Sheet-Metal Bending and Press-Brake Cells

Robotic press brake sheet metal bending has gained attention in industry due to the increasing demand for high productivity and consistent bending quality. In such systems, industrial robots are capable to handle sheet metal workpieces throughout the bending process, including gripping, positioning

at the press brake for bending process, reorientation for subsequent bending sequences and unloading process after the process completion. Compared to manual bending, robotic system reduces the operator's effort while ensures precise bending quality in a repeatable and reliable way. [1] Early research focused on analyzing feasibility of integrating industrial robots with traditional press brakes systems. These studies demonstrate that the sheet metal bending can be effectively performed when robot waypoints are carefully coordinated with the bending operation sequences. These results highlighted the importance of reliable synchronization between the robot and the press brake, particularly for precise process execution and accurate motion planning. [2]

Later studies of robotic sheet metal bending processes by researchers addressed the problem of process plaining, including the determination of bending sequences, selection of which tools need to be used for the specific bend and handling strategies for the workpiece. As a result, they showed bending depends on the geometry, reachability to process the bends. So, they found out that, make planning to make aware of unstable grasps or robot motions that cannot be executed during the process. [7] Unlike the rigid workpiece, the sheet metal continuously changes the shapes after each bending operation. This requires the robot to adjust the gripping position and adopt different orientational pickup through the process. To ensure safety, many robotic systems rely on conservative motion planning and fixed handling strategies. While this approach reduces the system complexity when dealing with the complex or varying part geometry. [1], [2] On the next stage, regripping becomes one of the most challenging parts for robotic bending. Often the sheet must be temporarily placed at an intermediate location and then repick again for the next bend, requiring appropriate regripping positions that are compatible with the bending tools. most existing systems rely on predefined regripping locations and manually planned regripping strategies. This significantly increases engineering effort and make system adaptability difficult for frequent process changes . [2], [7].

Research on path planning for robot assisted sheet metal bending shows constraints on their trajectories around press brake and tooling represents a major challenge. The waypoints must be carefully review in between the workspace especially during the reorientation and regripping operations [9].

Additional studies highlighted the role of grasp feasibility and stability, especially when vacuum based gripper are used. Research on vacuum gripper modelling and automatic grasp positions strongly depend on robustness and efficiency. [10]. However, these approaches tailored to specific de-

signed gripper types or handling strategies, and they are integrated into an automatic program generations press brake industrial applications

In industrial practice, robotic press brake bending performs well when the part geometry, tooling and process parameters are predefined. Such system typically relies on expensive offline preparation, accurate digital model simulations, manually defined handling and regripping strategies. However, as product variety increases, system flexibility significantly decreases. Consequently, the robotic press brake bending is well suited for structured and repetitive process. Nevertheless, current system still depends heavily on predefined process knowledge, pre planned trajectories and manually configured regripping strategies. These limitations reduce adaptability and have motivated extensive research into automated robot program generation methods that can respond to changing processes and production conditions. [1], [7].

2.5 Summary of the State of the Art and Identified Research Gap

The research has shown that the evolution of robot programming has evolved overtime. Early approaches use manuals teach pendant programming, followed by offline programming and recent development of automated and rule-based programming methods. Manual programming and offline programming are still widely used due to the robustness and direct physical interaction offered by programming. However, these methods require expert knowledge and therefore provide limited flexibility in industry. Offline programming optimizes some of these limitations of manual programming by enabling virtual simulation, but it remains limited by accuracy of digital model and need for manual defined task sequences. Automate rule-based program generation methods aims to reduce engineer efforts by enabling higher level system process which increases the consistency and modularity in robot programming. Nevertheless, these approaches rely on predefined rules and conservative planning assumptions, which restrict their applicability in highly variant production scenarios.

In the context of sheet metal bending and press brake automation, these limitations are prominently highlighted. The regripping, reposition workpiece for multiple bend on different sides significantly affects the reachability and grasp feasibility. Current robotic system therefore depends heavily on manually planned regripping strategies and pre-defined trajectory planning. As a result, adaptability to new part geometry remains limited, bending sequences lack flexibility and systematic reprogramming is still required when

process conditions change. Consequently, there exists a gap between existing robot programming approaches and the requirements of flexible robot sheet metal bending applications. There is a lack of methods that enable automatic program generations capable of addressing these constraints, reducing programming time, and allowing non experts operators to configure and execute bending processes. Addressing this gap is essential for increasing flexibility and adaptability in modern robotic industry

3 System Concept and Methodology

3.1 Problem Definition and Objectives

The state of the art has shown clearly that industrial robots are widely used for the press brake sheet metal bending, primarily through manual programming or predefined programming methods. These methods perform well in structured scenarios with fixed part position, geometries, and bending sequences. However, as product variants increase, the adoption of such robot programming approaches requires manual reprogramming and time consuming.

In press brake bending, after each bending operation, the shape of the sheet metal changes. Each geometric change affects robot reachability, gripping feasibility and grip point calculations, motion trajectories. Consequently, handling and regripping should be planned carefully. Previous research shows that managing this constraint is one of the most time-consuming and complex aspects of robotic bending processes particularly when handled manually.

From a methodological perspective, the limitations of this robot programming approaches related to their low-level nature. programmers must define the robot paths, position of tools and individual waypoints. This makes the programming, tightly coupled to a specific robot, grip positions or tool setups and making the reorientation or regripping strategies difficult to adapt. Robotic literature clearly indicates that such low-level programming does not scale well as system complexity increases and flexibility requirements grow [1]. This observation motivated research into the higher-level representation, where motion execution and decision making integrated with a unified framework. These methods aim to simplify handling constraints and promote reusable robot programs. While these strategies have been implemented in other robotic applications, they have not yet been implemented in robotic press brake sheet metal bending, particularly when the position and geometry changes must be explicitly considered.

Based on these observation , the central problem addressed in this thesis can be defined as follow: There is a lack of a systematic approach for automatic generation of robot programs for press brake sheet metal bending that explicitly accounts for changing workpiece positions and geometry , handling feasibility and regripping requirements while minimizing the man-

ual programming .

The primary objective of this thesis is to develop a conceptual and methodological framework for the automatic generation of the robot program that integrates bending, handling and regripping operations in one workflow. This proposed method aims to reduce manual programming effort and improve adaptability to different part geometries. To achieve this objective ,the work focuses on integrating bending sequences , handling actions , opposite side bending and regripping operation and which are validated and verified in simulation environment compatible with industrial robot environment .The scope of the thesis is limited to virtual environment and conceptual evaluation.

3.2 Requirement Definition and Management

To ensure a systematic and verifiable development of the proposed automatic robot program generation approach, the system design is guided by explicitly defined requirements. These requirements turn the research gap and problem formulation into engineering objectives that can later be verified and validated. The requirement that are explained in this chapter are directly derived from the limitations identified in the state of art and from the objectives formulated in 3.1. Rather than describing implementation details , the requirement specify what the system must achieve under which condition it must operate .This separation ensures that design decision remain traceable and that verification activities can be performed objectively. The requirement are classified into functional and non-functional requirements .Functional requirements describe the expected system behavior and capabilities , while nonfunctional requirements define the quality attributes such as usability , robustness and safety .In addition , system constraints and assumptions are explicitly stated to scope of the work and to define the operational boundaries of the proposed solution To support systematic verification, each requirement is formulated to Specific measurable Achievable Relevant Timebound (SMART) principle. Furthermore, each requirement is linked to system components and verifications steps. This traceability ensures that all requirements can be checked later and that the development process remains transparent and consistent.

3.2.1 Functional Requirements

Functional requirements specify the capabilities that the system provides in order to address the problem of automatic robot program generation for press brake sheet metal bending. These requirements define the expected system behavior from the task execution, operator interaction, trajectory generation and recovery mechanism. At a high level, the system shall enable the automatic generation of robot programs for multi-step sheet metal bending process without requiring manual t programming. The system shall support the execution of bending sequences that involve changes in workspace geometry, orientation, handling and regripping strategies.

A central functional aspects of the system is the consideration of work piece geometry evolution during bending process .Since the shape , position , orientation of the sheet metal changes after each bending operations , the system shall be feasible to handle the current geometric state and position of the workpiece when generating subsequent actions for the next task. This requirement is essential to ensure feasibility with respect to robot reachability and tool operations throughout bending sequence. Furthermore , the system shall support bending operations that requires for different sides of the work piece .Opposite side bending introduces additional constraint on tool orientation and robot configuration , which cannot always be resolved with a fixed handling strategy and time consuming .The system program therefore capable of adapting this strategy , including workpiece orientation and grip configuration, to enable such operation while maintaining feasibility

To structure the functional requirements, they are grouped into categories.

Operator Interaction Requirements: The system shall allow limited operator interaction for configuration of robot and process selection while minimizing the manual intervention during the execution. Before execution, the operator shall be able to specify the required process output, such side for bending or cad model for bending. During execution, the robot operation shall provide clear feedback or warning message when the given output conditions are unsafe for the process. The role of operator shall be restricted to configuration, while the program generation logic shall be handled automatically by the system.

Data Handling Requirements: The system should automatically use the operator input data and process the program that need to processed for

the required output in a consistent and reliable manner .This includes Handling geometries , grip position calculation, tracked poses , configuration parameters and intermediate state information generated during the time of execution. Data shall be used consistently across different stage of the program generation process to ensure that decisions regarding the handling and orientation and regripping are based on the current system state.

Motion Execution Requirements: The system shall generate feasible robot waypoints for all handling and bending related operations. This includes generation of approach waypoints, placement motions and the execution of orientation changes of the work piece and tool. Trajectory plans should be calculated prior to execution to prevent unreachable target poses. In addition, the system shall determine and apply appropriate grip positions on the workpiece to support handling and picking from different bending configurations. For opposite side bending operations, the system shall identify the grip position and waypoints such that required tool alignment can be achieved without violating the kinematical limits. Grip selection shall be treated as an integral part of the program generation process rather than manual integrated adjustments.

Error Handling and Regripping Requirements: The system shall be capable of detecting infeasible target poses or unsafe execution conditions before execution. When such conditions are detected, the system shall initiate a warning message to the user, trigger appropriate recovery actions instead of continuing execution. Recovery actions may include regripping or repositioning the workpiece to restore reachability. Regripping operation shall be supported as functional capability of the system .The system shall be able to integrate regripping sequence into generated robot program when direct trajectories to the target are unreachable or unsafe .If a critical error cannot be resolved through recovery actions , the system shall terminate execution in controlled regripping or repositioning manner to prevent unsafe robot behavior.

3.2.2 Non-functional Requirements

NFR defines the quality attributes and operation characteristics that elaborates how well the system performs its functions. While the functional requirements specify the system capabilities, non-functional requirements describe constraints related to usability, maintainability, robustness and safety.

Table 3.1: Functional Requirements for Automatic Robot Program Generation

ID	Functional Requirement (SMART)	Acceptance Criteria	Verification Method
FR-01	The system shall detect unreachable target poses before robot motion execution.	The robot shall not execute motion to unreachable poses and shall provide an error message.	Simulation Based
FR-02	The system shall allow operator-side selection prior to program execution.	The selected bend side shall be applied correctly to the generated robot program.	Simulation Based
FR-03	The system shall execute a regripping sequence when a target pose is unreachable within the current bend side.	Regripping shall be performed successfully and execution shall continue without interruption.	Simulation Based
FR-04	The system shall handle workpiece orientation changes during program execution.	Orientation changes shall be applied correctly without violating kinematic limits.	Simulation Based
FR-05	The system shall support opposite-side bending operations when required.	The system shall correctly execute opposite-side bending operations.	Simulation Based
FR-06	The system shall determine and apply suitable grip positions for each workpiece state.	Grip positions shall be determined and applied correctly for each workpiece state.	Simulation Based
FR-07	The system shall terminate execution when an unsafe operation is detected.	The robot shall terminate execution and display an error message.	Simulation Based

These aspects are important for automated robotic systems, where corrected system behavior alone is not sufficient if the system is difficult to use, fails under complex variations or behaves unsafely during execution. The NFR of the system are derived from the operational context of press brake sheet metal bending, where industrial robot operates in close proximity to tooling, workpieces and cell boundaries. The system is intended to support flexible manufacturing scenarios with minimal operator programming while maintaining the user conditions and safe behavior. To ensure clarity, the NFRs are grouped into the following categories.

Usability Requirements: The system should be designed such that the user can operate the system easily by minimizing the operator interaction during normal operation. Configuration steps required prior to execution shall be limited to essential parameters and shall be presented in a clear and understandable manner. The system should give meaningful feedback to operator, including warning information and error messages when unreachable waypoints or unsafe condition are detected. These messages shall support the operator in understanding the cause of an issue without requiring expert knowledge of robot programming.

Maintainability Requirements: The system shall support maintainability by enabling configuration changes without requiring modifications to the source code. Process parameters, handling strategies and system setting shall be adjustable through configuration mechanisms that are separated from the core program logic. In addition, the system should provide sufficient logging or terminal output to support debugging, monitoring and future extensions. These measures are intended to facilitate adaptation of the system to new workpiece, process variations, with minimal programming effort.

Robustness Requirements: The system shall exhibit robust behavior under repeated execution cycles. Small variations in workpiece, orientation or exceptions timing or path shall not lead to system failure or unsafe behavior. The system shall handle such behavior deviation in controlled manner, ensuring smooth operations whenever feasible or triggering appropriate recovery actions when required. Robustness is particularly important in flexible manufacturing environments where idealized conditions cannot be assumed.

Safety Requirements: The system should always ensure safe operations by validating generated robot motion trajectories, informing the user about unsafe conditions and enforcing defined safety constraints and forbidden zones to prevent unreachable or unsafe poses during program execution. In the events of critical errors or unsafe conditions that cannot be resolved through recovery actions, the system shall terminate execution in a controlled manner.

3.2.3 System Constraints and Assumptions

In addition to functional and non-functional requirements, the development of the proposed automatic robot program generation system is subject to a defined sets of constraints and assumptions. These Constraints delimit the scope of the thesis and clarify the operational boundaries within which the system is designed to function. Explicitly stating these limitations ensures transparency and prevents misinterpretation of the intended system capabilities. The proposed approach is developed for a predefined robotic press brake cell configuration. The mechanical layout of the robot, press brake, tooling and auxiliary components is assumed to remain fixed throughout the system operations. Changes to physical robotic position and tool geometry, and non-collision movement planning are out of scope of this work. The system assumes the availability of accurate geometrical models of the robot, workpiece, press brake, and related components within the virtual environments. The correctness of reachability analysis and motion feasibility check depends on the validaty of these models. Model inaccuracies, calibration errors or unmodeled physical effects are not explicitly compensated within the proposed framework. Explicitly defining these limitations ensures transparency and prevents over interpretation of the system capabilities. The constraints are grouped into geometric, process related, cell layout, handling and safety limitations.

Sheet geometry Constraints : The system is designed for rectangular sheet metal workpiece only. All geometric reasoning, grip position calculations and edge distance check assumes a rectangular shape with four orthogonal edges. Nonrectangular geometries, polygonal shapes or free form contours are not supported. The sheet geometry is parameterized by its length, width thickness and mass. Optional circular holes may be defined using radial parameters. However, only circular holes are supported, irregular polygonal cuts are not considered. All hole's related checks are per-

formed using radial distance evaluation. Furthermore, only a single global wall height parameter is considered during the handling. Per edge height customization is not supported.

Bend and process constraints: The bending process is restricted to 90 degrees of bend angles. Variable user defined bend angles are not supported. The bending logic assumes one bend per edge and each edge can be associated with at most one bending operation. A maximum of four bends per side is supported. For opposite side operation, the number of bends is similarly limited to four. The program logic prevents the duplicate bend assignments and multiple bend assignments on the same edge. This ensures a consistent one bend per edge configuration and simplifies the geometric reasoning required for the automatic program generation.

Robot and cell layout constraints: The robotic cell configuration is assumed to be fixed. The robot base, press brake, fixtures and auxiliary components are defined relative to a constant world coordinate frame. Relocation or reconfiguration of these elements invalidates the area logic, reachability assumptions and orientation constraints. In this case, system requires manual updates of configuration values for adaptation. Tool configuration is also assumed to be fixed. The system operates with a predefined TCP (tool center point) and attachment frame (Flange) Changes tool geometry, TCP definition or IO wiring require configuration updates and may necessitate software adaptation. Vacuum gripping is controlled via single digital output. Alternative control schemes are not considered.

Gripping constraints: The system is designed exclusively for vacuum based gripping. Grip feasibility checks are based on physical considerations including pressure difference friction and rated shear force. The holding capability of a vacuum gripper depends primarily on the achievable pressure difference between suction cup and the ambient atmosphere, the effective contact area and the frictional interaction between workpieces surfaces and the suction elements.[10]. Mechanical magnetic or finger-based grippers are not supported within the current framework.

Safety and warning system constraints: The safety and warning logic monitors only a predefined ser of fixtures and reference objects within the

cells. Full collision detection, dynamic safety trajectory planning and advanced real time safety validation are not implemented. Safety evaluation is limited to predefined geometric checks and rule-based constraints within virtual environment. Object tracking and naming assumptions -The systems rely on predefined object identifiers within the virtual environment. Specific object names are expected as key to finding to process or finding the objects. Renaming or removing these objects disrupts the workflow logic and may lead to execution failure. Therefore, renaming objects requires manual updates in the program configuration.

3.2.4 System Traceability

Requirements Traceability is a fundamental principle in systematic engineering development, ensuring that all defined requirements are consistently reflected in the system design, implementation and validation stages. In the context of the proposed automatic robot program generation system, traceability guarantees that the transition from conceptual objectives to executable functionality remains transparent, structured and verifiable. Each requirement is defined in section 3.2.1 (functional requirement) and Section 3.2.2(Nonfunctional requirements) is assigned to a unique identifier (e.g. FR-01, NFR -019:) .These identifiers serve as persistent reference throughout the entire development lifecycle. The identifiers are used consistently in the following aspects:

- Every requirement is implemented within a specific module.
- Every requirement is verified through a defined test scenario.
- No requirement remains unnoticed.
- No implementation functionality exists without requirement justification.

Forward traceability: confirms that every requirement is realized in a specific system components or logic module.

Backward traceability: confirms that every implemented functionality can be tracked back to an explicitly defined requirement.
By mapping each requirement to a corresponding implementation module and validation scenario , the development process become auditable and reproducible, which is particularly important in systems involving automated

decision making , such as traceability evaluation , regripping logic , and opposite side handling .In addition tractability improves maintainability and future extensibility, such as adapting to a new gripper type or extending bend angle support . This structure allows systematic impact analysis. Affected requirements and modules can be identified quickly, thereby reducing development risk.

Table 3.2: Requirement Traceability Matrix

Requirement ID	Requirement Description	Implemented In	Verified In
FR-01	Detect unreachable target poses before execution.	Reachability check module	Simulation test case TC-01
FR-02	Allow operator-side selection before execution.	User interaction logic	Execution test case TC-02
FR-03	Execute regripping sequence when required.	Regrip handling module	Regrip validation test case TC-03
FR-04	Handle orientation changes during execution.	Orientation management logic	Orientation validation test case TC-04
FR-05	Support opposite-side bending workflow.	Opposite-side runner module	Opposite-side validation test case TC-05
FR-06	Terminate safely on critical errors.	Safety monitoring and error-handling module	Safety validation test case TC-06
NFR-01	Ensure minimal operator interaction.	Dialog configuration logic	Usability validation
NFR-02	Allow configuration without source code modification.	Configuration management module	Configuration validation test
NFR-03	Provide clear warning and error messages.	Warning generation module	Warning message validation
NFR-04	Ensure robustness under repeated execution cycles.	Execution loop logic	Repeated run validation test
NFR-05	Tolerate moderate pose deviations within defined limits.	Pose validation logic	Perturbation validation test

3.3 System Architecture and Design

The system architecture is directly derived from the functional and nonfunctional requirements defined in section 3.2. Its purpose is to transform the sheet metal bending program inputs into executable robot programs while ensuring deterministic behavior and modular extensibility. Each architecture layer performs a clearly defined responsibility in the transformation chain from high level task specification of robot motion execution. Separation of responsibility ensures traceability, maintainability and controlled evolution. The architecture is composed of five main conceptual layers:

3.3.1 Input and configuration layer

This layer serves as the formal interface between operator defined process parameters and internal system logic. It collects all necessary geometric and process related information required for programming generation, including sheet dimensions, bending sides and configurations parameters for executable programming. This layer performs preliminary validation to ensure parameters completeness and consistency before forwarding structured data. It checks whether all required parameters are provided, whether the values are complete and whether the data is logically consistent, thereby preventing errors in the later stages of the process. The input layer is separated from the core design logic of the system, meaning that user interaction (configuration) is independent of the robot execution logic. Changing configuration parameters does not directly affect the system structure. This supports maintainability and minimizes unintended side effects when configuration parameters are modified. The output of this layer is a structured task representation describing the desired bending operation and associated handling operations.

3.3.2 Geometry and Constraint Evaluation Layer

The geometry Evaluation layer is responsible for interpreting the workpiece model and applying geometric feasibility rules which include, Identification of sheet edges and their orientation are related to the global coordinate system, determination of candidate grip positions based on geometry, Validation of edge distance and hole clearance and Verification of compatibility with defined wall height and spatial constraints. This layer ensures that all the operations comply with the geometric assumption defined in section 3.2.3 only geometrical valid configurations are forwarded to process the pro-

gram. If a violation is detected, execution is halted and warning message is generated before the motion generation begins. The valuation process is deterministic, and rule based, avoiding heuristic or probabilistic decision mechanisms. This ensures reproducibility and simplifies validation.

3.3.3 Planning and Decision Logic layer

The planning and decision logic layer determines the logical sequence of operations required to achieve the requested bending configurations on reachability and geometric feasibility, it decides:

- Whether a bend can be executed directly
- Whether intermediate regripping is required
- Whether opposite side handling logic must be invoked
- The ordering of sequential bending operations

The decision logic implements conditional rules derived from the functional requirements. It does not generate low level trajectories but defines a high-level execution sequence consisting of discrete operational states. This separation between decision logic and motion generation enhances clarity and simplifies system debugging. Changes in bending or regripping strategy can be implemented at this level without modifying lower-level motion structures.

3.3.4 Motion generation layer

This motion generation section transforms all the planned operation sequences into executable robot command, it defines, Target poses for gripping and placement, Orientation transformation for bend execution, Intermediate transfer positions and TCP alignment with workpiece reference frame. Motion generation operates under the assumption of a fixed world coordinates frame and predefines tool configuration. All transformations are determined and derived from geometric and process inputs. This layer also ensures the logical trajectories planning while respecting robot kinematics and spatial constraints.

3.3.5 Safety and validation layer

This layer acts as the final verification stage before the motion executes. It performs the reachability verification, evaluation of forbidden zones, con-

straints re checking prior to execution-controlled termination in case of unfeasible or unsafe states. By isolating safety logic in a dedicated layer, the architecture ensures that safety validation remains independent of planning and motion algorithms, decision making logic and motion computation, which ensures the safety rules are always applied and it supports compliances with NFR such as robustness and operational safety.

3.3.6 Architectural Rationale

The chosen layered structure ensures a clear separation between configuration handlings, geometric reasoning, planning logic and motion execution. This separation enhances transparency and reduces the coupling between components, thereby supporting maintainability and modular refinement. The deterministic and rule based nature of the architecture ensures predictability system behaviors and structured decision making, which is an important aspect for industrial environments .In addition the explicit mapping between architectural components and defined FRs enables direct traceability and systematic verification .Finally the structure supports controlled extensibility and allows future enhancement such as additional gripping strategies or extending bending capabilities to be integrated without fundamental redesign of the system architecture .This architecture intentional avoids adaptive and learning approaches , thereby prioritizing reliability and predictability execution in the context of robotic press brake automation.

3.4 Algorithmic Implementation and Workflow

Building upon the layered system architecture introduced in section 3.3, this section formalizes the algorithmic workflow that transform validated configuration inputs into executable and constraints compliant robot motion sequences. The workflow represents the operational realization of the FR and NFR defined in section 3.2. Its structure is strictly hierarchical, geometric validation precedes grip determination, grip validation precedes reachability assessment, and reachability evaluation preceded motion generation. This ordering is deliberate and ensures that no invalid state can propagate towards execution. An overview of the workflow is provided in Figure 3.1, with further details discussed in the subsequent sections.

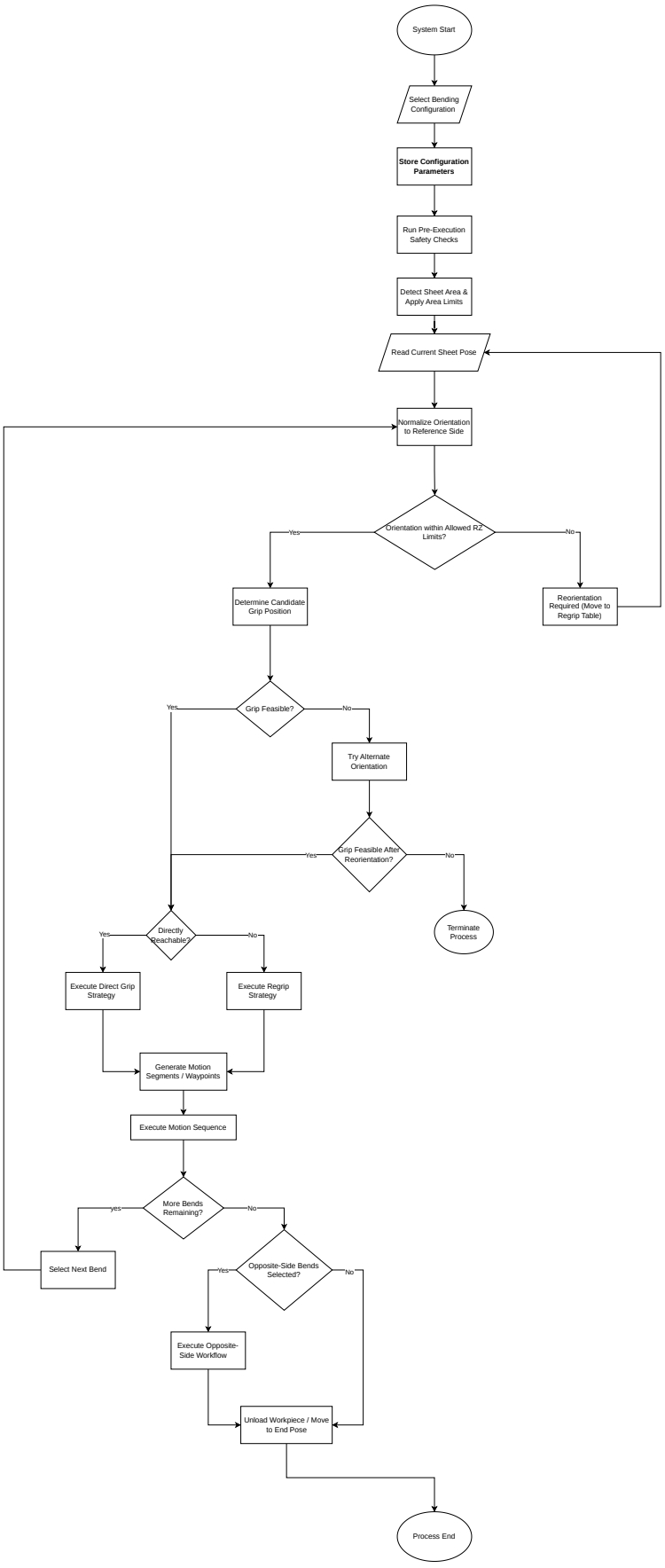


Figure 3.1: System architecture of the automatic robot program generation framework

3.4.1 Initialization and configurations validation

The workflow begins with structured configuration section. At this stage, the bending sequence and associated parameters are defined. This configuration stage establishes the deterministic input boundary of the system. The purpose of this stage is to ensure completeness and internal consistency of operator defined parameters before geometric verification begins. By enforcing configuration validation at the outset, the system satisfies the requirement that execution shall not begin under incomplete or ambiguous input conditions.

Following configuration stage, a pre-execution validation stage is performed. This stage verifies required objects and fixtures that are available in the workspace. The rationale behind this step is to isolate environmental failures before geometric planning occurs. This design decision guarantees compliance with robustness and safety requirements by preventing downstream computation on invalid environmental assumptions.

Subsequently, the areas where the sheet is identified and corresponding spatial constraints are applied. The reason for performing area classification at this stage is to ensure that orientation and reachability check operate within the context dependent boundaries. By adapting limits to the detected area, the system ensures that subsequent decision stages evaluate feasibility relative to the correct operational region.

3.4.2 Pose Acquisition and Orientation Normalization

After initial validation, the system retrieves the current pose of the system from the tracking subsystem. The retrieved pose reflects the measured position and orientation in the global coordinates frame. However, the original pose from the tracking subsystem alone is insufficient for the consistent geometric reasoning because it may not align with the defined bending reference. Therefore, the orientation is normalized relative to a predefined reference side. This normalization is to eliminate ambiguity in geometric interpretation and to ensure that subsequent reasoning is performed within a consistent frame of reference. This design always guarantees reproducibility across bending steps and prevents inconsistent interpretation of identical geometric states.

The normalized orientation is then evaluated against predefined rotational constraints. These constraints represent the spatial and kinematic limitations of the robotic cell. The validation step exist to ensure that planning decisions are based on physically admissible states .If the orientation vio-

lates the allowed limits of the robot , a reorientation procedure is triggered .The workpiece is repositioned and pose acquisition is repeated .This loop ensures that no planning is made under invalid orientation conditions .By enforcing this validation hierarchy , the system prevents unstable geometric states from influencing grip or motion generation.

3.4.3 Grip Position Determination and Validation

Once orientation is valid and has been established, the workflow proceeds to grip determination. Grip positions are computed based on the geometric boundary conditions and predefined constraint rules. The objective of this stage is to identify the position to grip by satisfying spatial clearance and conditions.

Grip feasibility evaluation is performed prior to reachability assessment's ordering is intentional because a grip configuration that violates geometric constraints must be rejected before evaluating whether the robot can reach a bending pose. This validation layer prevents errors and ensures that reachability is never evaluated for an invalid handling state.

If the initial grip configuration violates defined constraints, and alternate orientation is evaluated. Grip feasibility is re assessed under the modified orientation. If feasibility cannot be achieved even after the reorientation, the workflow transition will undergo controlled termination. This explicit termination condition serves as proof that the system enforces safety and does not attempt to execute undefined or unstable handling states. The termination branch directly satisfies the safety requirement that invalid configuration shall not propagate to execution.

3.4.4 Reachability Assessment and strategy selection

After confirming the grip feasibility, the system evaluates the bending pose is reachable with kinematics and orientation constraints. The purpose of this stage is to analyze whether the robot operation can be executed directly or require intermediate repositioning.

If the target pose is reachable, the direct execution is triggered. This choice minimizes the intermediate handling and reduces the process complexity. If in case of target pose not reachable, a regrip strategy is then triggered. The regrip strategy introduces an intermediate repositioning stage that transforms the sheet into a compatible position with reachable limits. The decision between direct and regrip strategies is automated based on the

sheet tracking position and orientation. No optimization and adaptive mechanism are involved. The convergence of both strategies into a unified motion generation stage ensures structural consistency of the execution methods. This design ensures that the trajectory generation is performed only after full validation of grip and reachability conditions, which reduces unwanted trajectory of robot.

3.4.5 Motion generation and execution

Following strategy selection, motion segment and intermediate waypoint are generated. These waypoints are generated exclusively from the validated geometry and kinematics states. The system does not introduce ethe new geometric assumptions at this stage; it translates previously validated decisions into executable robot trajectories. Execution occurs only after completion of all preceding validation. This ordering ensures compliance with robust requirements by guaranteeing that every executed motion originates from a constraint complainant state. The deterministic generation ensures reproducibility, transparency, easier debugging and verification.

3.4.6 Iterative Multi bend processing

After completion of the one bending operation, the workflow evaluates whether bends remain in the selected sequence. This will evaluate to maintain full validation integrity across multi step operations. If further bend is required, the next bending operations are selected, and the returns to normalization stage again and undergoes again the validation, including orientation verification, grip feasibility assessment and reachability analysis. This design ensures each bend undergoes complete validation. By reapplying this full validation hierarchy for each bend, the system prevents accumulation errors, keeps the geometry consistent and guarantees controlled state transition throughout multistep execution.

3.4.7 opposite side handling and controlled completion

Upon completion of all primary bending operations , the workflow evaluates whether opposite bend side has been selected .If such operations are selected for bending , a different execution phase is initiated under the same validation hierarchy .orientation normalization , grip feasibility and reachability check are enforced consistently .along with that, for opposite side bending , a separate trajectories and operation are triggered to flip the sheet to make

the opposite side bending process. After this process the regrip station with dedicated loops are executed according to the bend side. After all bending tasks have been completed, the workpiece is transferred to a predefined end pose and unloaded. The system then transitions to a controlled terminations state. The explicit separation between execution and termination ensures that the process terminates in a safe configuration.

3.4.8 Deterministic and constraint driven proof of correctness

Described workflow is entirely deterministic and constraint driven. Cah processing stages enforces validation prior to execution derived from the system requirements. The hierarchical validation structure guarantees that geometric inconsistencies, infeasible grips or unreachable poses are detected before ethe program execution.

This structure validation constitutes methodological proof that the system operates in a logically consistent manner. By ensuring that executed motion originates from a verified and constraints compliant state, the workflow satisfies the FR related to correct waypoint generation, as well as NFR concerning predictability and safety. The absence of adaptive learning-based components further strengthen reproducibility and transparency, which are essential characteristics in industrial robotic press brake automation.

3.4.9 State Management and Object Tracking

The workflow relies on a consistent state management system that tracks the current configuration of the workpiece, including its position, orientation and grip state. This state information is updated at each stage of the workflow and is used to inform decision making in subsequent stages. Object tracking is performed using predefined identifiers within the virtual environment, ensuring that the system can reliably reference the workpiece and related fixtures throughout the execution process. This structured state management supports traceability and ensures that all operations are based on accurate and up-to-date information about the system’s current state.

4 System Implementation and Integration

4.1 Planning and constraint Module Implementation

This chapter presents the technical realization of the constraints driven system architecture and algorithmic workflow introduced in chapter 3. The previous chapter elaborated the methodological design of the system, the present chapter demonstrates how this methodology is implemented with an industrial robotic automation environment and convert the manual programming to automated programming robotic automation. The objective of this section is to define the development context, system boundaries, and integration structure within which the automatic robot program generation system operates.

The implementation is used inside an industrial robotic press brake cell. Such environment requires coordinated interaction between geometric reasoning, motion planning, robot control and peripheral device handling. Therefore, the system must not only execute validated motion sequence but must also ensure consistent synchronization between computational planning and physical robot execution.

From the system engineering perspective, the implementation is structured into three principal layers, a planning layer, an integration layer, and an execution layer. The planning layer contains rules based which are responsible for the orientation validation, grip determination, reachability assessment and strategy selection. The integration layer contains communication between the planning system and the robot control environment. The execution layer translates and validates the waypoint and functions to the physical robotic actions.

The separation between planning and execution is intentional and flows established modular automation principle, where decoupling decision – making from actuation improves system reliability and maintainability [11]. By maintaining this separation, the system ensures that constraints validation remain logically independent from motion execution. This ensures no motion is generated without geometric validation, grip stability and reachability conditions. The implementation environment provides motion primitives, coordinate management, and hardware abstraction through a controlled robotic framework. The planning system interfaces with this environment through a structured communication mechanism which enables the transfer of val-

idated motion path. This structure allows automatic generation of robot programs with manual waypoint teaching, thereby replacing conventional teach pendant programming approaches discussed in Chapter2.

Simulation capability is incorporated with the implementation environment to enable pre-execution validation of all the decided trajectories. Simulation based validation and virtual commissioning are widely recognized as essential components of modern industrial robotic system deployment, as they allow execution logic to be verified prior to physical operation [12]. The integration of simulation into development workflow therefore supports robustness and reduces commissioning risk. The implemented process structured configuration inputs transformed them into validated geometric states and generated executable motion sequences through rule evaluation. Every execution is preceded by structural validation stages as defined in 3.4. This layered program ensures compliance with FR related to correct motion generation and NFR concerning predictable behavior. The overall integration context of system is illustrated in Fig 4.1, which shows the communication flow between the planning system and the robot control environment. The planning system generates validated motion paths based on the current state of the workpiece and the defined constraints. These paths are then transmitted to the robot control environment, which executes the corresponding robotic actions while ensuring synchronization with peripheral devices and safety monitoring systems.

From a development standpoint, the system is implemented with emphasis on execution and explicit state control. No learning-based mechanisms are used, while all the decisions are governed under constraint conditions. This design simplifies validation in an industrial automation context where predictable behavior is mandatory. In summary, this section established the technical integration framework within which the automatic robot program robotic system operates. The following section details the realization of individual functional components, beginning with the robot control environment and communication interface.

4.1.1 Layered Software Architecture Realization

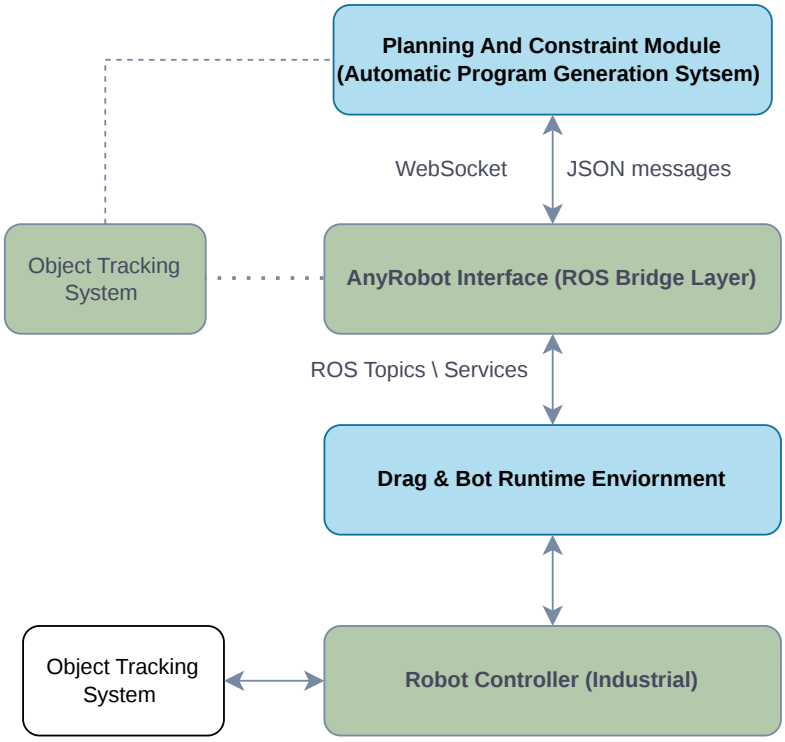


Figure 4.1: Integration of planning system with AnyRobot Interface

Bibliography

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, 2nd ed. London: Springer, 2016, ISBN: 978-1-4471-1500-0.

[2] S. Aomura and K. Koguchi, "Development of a robotic bending system for sheet metal," *Journal of Materials Processing Technology*, vol. 130–131, pp. 362–367, 2002. DOI: 10.1016/S0924-0136(02)00744-1

[3] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 20, no. 2, pp. 137–150, 2012. DOI: 10.1016/j.rcim.2003.07.004

[4] J. Krüger, T. K. Lien, and A. Verl, "Cooperation of human and machines in assembly lines," *CIRP Annals*, vol. 58, no. 2, pp. 628–646, 2009. DOI: 10.1016/j.cirp.2009.09.009

[5] C. Schou, J. S. Damgaard, S. Bøgh, and O. Madsen, "Human–robot interface for instructing industrial robots using skill primitives," *Robotics and Computer-Integrated Manufacturing*, vol. 29, no. 6, pp. 403–412, 2013. DOI: 10.1016/j.rcim.2013.06.002

[6] A. Björkelund, J. Malec, K. Nilsson, and P. Nugues, "Knowledge-based industrial robot task planning and execution," *Robotics and Computer-Integrated Manufacturing*, vol. 28, no. 5, pp. 558–568, 2012. DOI: 10.1016/j.rcim.2012.01.003

[7] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998. DOI: 10.1016/S0278-6125(98)80035-2

[8] Q. Glorieux, J. Vierendeels, and J. De Schutter, "Quality- and productivity-driven trajectory optimisation for robotic handling," *Robotics and Computer-Integrated Manufacturing*, vol. 54, pp. 37–49, 2018. DOI: 10.1016/j.rcim.2018.05.001

[9] X. Liao and G. G. Wang, "Evolutionary path planning for robot-assisted sheet metal bending," *Journal of Manufacturing Systems*, vol. 22, no. 2, pp. 112–127, 2003. DOI: 10.1016/S0278-6125(03)80006-4

[10] J. R. Duflou, D. Van Oudheusden, J.-P. Kruth, and D. Cattrysse, "Modeling of vacuum grippers for the design of energy-efficient vacuum-based handling processes," *CIRP Annals*, vol. 55, no. 1, pp. 17–20, 2006. DOI: 10.1016/S0007-8506(07)60375-0

[11] K. Thramboulidis, "Model-driven engineering for industrial automation systems: A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 1893–1903, 2013. DOI: 10.1109/TII.2013.2251233

[12] M. Bär, G. Reinhart, and J. Schilp, "Virtual commissioning of robotic systems: A review and outlook," *Procedia CIRP*, vol. 72, pp. 129–134, 2018.